

ПОВНА ДОКУМЕНТАЦІЯ ВСІХ FEATURES ДЛЯ COFFEE RECOMMENDATION SYSTEM

Total Features: ~147

ЗМІСТ

1. Basic Taste Features (8)
 2. Taste Difference Features (6)
 3. Advanced Taste Match Features (5)
 4. User Aggregate Features (6)
 5. Recipe Aggregate Features (5)
 6. Equipment Features (12)
 7. Technical Match Features (5)
 8. Tag and Category Features (9)
 9. Dietary Features (5)
 10. Historical Features (13)
 11. Cold Start Features (2)
 12. Basic Interaction Features (3)
 13. Temporal Features (9)
 14. Behavioral Features (6)
 15. Alignment Features (2)
 16. Advanced Cross Features (47)
 17. Miscellaneous (1)
-

1. BASIC TASTE FEATURES (8)

Базові характеристики рецептів з датасету.

1.1. taste_bitterness

Опис: Рівень гіркоти рецепту

Діапазон: 0.0 - 1.0

Джерело: recipes.csv

Код:

```
python
```

```
# Directly from recipes_df  
taste_bitterness = recipes_df['taste_bitterness']
```

1.2. taste_sweetness

Опис: Рівень солодкості рецепту

Діапазон: 0.0 - 1.0

Джерело: recipes.csv

Код:

```
python  
  
taste_sweetness = recipes_df['taste_sweetness']
```

1.3. taste_acidity

Опис: Рівень кислотності рецепту

Діапазон: 0.0 - 1.0

Джерело: recipes.csv

Код:

```
python  
  
taste_acidity = recipes_df['taste_acidity']
```

1.4. taste_body

Опис: Рівень тіла/насиченості рецепту

Діапазон: 0.0 - 1.0

Джерело: recipes.csv

Код:

```
python  
  
taste_body = recipes_df['taste_body']
```

1.5. strength

Опис: Міцність кави (кофеїн)

Діапазон: 1 - 5

Джерело: recipes.csv

Код:

```
python  
  
strength = recipes_df['strength']
```

1.6. `preparation_time_minutes`

Опис: Час приготування в хвилинах

Діапазон: varies (typically 5-30 minutes)

Джерело: recipes.csv

Код:

```
python

preparation_time_minutes = recipes_df['preparation_time_minutes']
```

1.7. `difficulty_numeric`

Опис: Складність рецепту (числовий формат)

Діапазон: 1 (beginner), 2 (intermediate), 3 (advanced)

Джерело: recipes.csv (converted)

Код:

```
python

difficulty_mapping = {'beginner': 1, 'intermediate': 2, 'advanced': 3}
recipes_df['difficulty_numeric'] = recipes_df['difficulty'].map(difficulty_mapping)
```

1.8. `portion_size_ml`

Опис: Розмір порції в мілілітрах

Діапазон: typically 150-500 ml

Джерело: recipes.csv

Код:

```
python

portion_size_ml = recipes_df['portion_size_ml']
```

2. TASTE DIFFERENCE FEATURES (6)

Різниця між смаковими перевагами користувача та характеристиками рецепту.

2.1. `diff_bitterness`

Опис: Абсолютна різниця в гіркоті між перевагою користувача та рецептом

Діапазон: 0.0 - 1.0

Чому важливо: Показує наскільки рецепт відповідає смаковим перевагам

Код:

```
python
```

```
df['diff_bitterness'] = abs(df['taste_pref_bitterness'] - df['taste_bitterness'])
```

2.2. diff_sweetness

Опис: Абсолютна різниця в солодкості

Діапазон: 0.0 - 1.0

Код:

```
python
```

```
df['diff_sweetness'] = abs(df['taste_pref_sweetness'] - df['taste_sweetness'])
```

2.3. diff_acidity

Опис: Абсолютна різниця в кислотності

Діапазон: 0.0 - 1.0

Код:

```
python
```

```
df['diff_acidity'] = abs(df['taste_pref_acidity'] - df['taste_acidity'])
```

2.4. diff_body

Опис: Абсолютна різниця в насиченості

Діапазон: 0.0 - 1.0

Код:

```
python
```

```
df['diff_body'] = abs(df['taste_pref_body'] - df['taste_body'])
```

2.5. diff_strength

Опис: Абсолютна різниця в міцності кави

Діапазон: 0 - 4

Код:

```
python
```

```
df['diff_strength'] = abs(df['preferred_strength'] - df['strength'])
```

2.6. is_strength_match

Опис: Чи збігається бажана та фактична міцність

Діапазон: 0 (no match) або 1 (match)

Чому важливо: Точний збіг міцності важливий для задоволення користувача

Код:

```
python

df['is_strength_match'] = (df['preferred_strength'] == df['strength']).astype(int)
```

3. ADVANCED TASTE MATCH FEATURES (5)

Просунуті метрики схожості смакових профілів.

3.1. `taste_cosine_similarity`

Опис: Косинусна подібність між смаковими векторами користувача та рецепту

Діапазон: -1.0 - 1.0 (зазвичай 0.0 - 1.0)

Чому важливо: Найважливіша метрика для персоналізації! Вимірює загальну схожість смакових профілів

Код:

```
python

def compute_taste_cosine_similarity(row):
    user_vector = np.array([
        row['taste_pref_bitterness'],
        row['taste_pref_sweetness'],
        row['taste_pref_acidity'],
        row['taste_pref_body']
    ])
    recipe_vector = np.array([
        row['taste_bitterness'],
        row['taste_sweetness'],
        row['taste_acidity'],
        row['taste_body']
    ])
    if np.linalg.norm(user_vector) == 0 or np.linalg.norm(recipe_vector) == 0:
        return 0
    return 1 - cosine(user_vector, recipe_vector)

df['taste_cosine_similarity'] = df.apply(compute_taste_cosine_similarity, axis=1)
```

3.2. `taste_euclidean_distance`

Опис: Евклідова відстань між смаковими векторами

Діапазон: 0.0 - 2.0

Чому важливо: Менша відстань = кращий збіг

Код:

python

```
def compute_euclidean(row):
    user_vector = np.array([
        row['taste_pref_bitterness'],
        row['taste_pref_sweetness'],
        row['taste_pref_acidity'],
        row['taste_pref_body']
    ])
    recipe_vector = np.array([
        row['taste_bitterness'],
        row['taste_sweetness'],
        row['taste_acidity'],
        row['taste_body']
    ])
    return euclidean(user_vector, recipe_vector)

df['taste_euclidean_distance'] = df.apply(compute_euclidean, axis=1)
```

3.3. taste_manhattan_distance

Опис: Манхеттенська відстань (сума абсолютних різниць)

Діапазон: 0.0 - 4.0

Код:

python

```
df['taste_manhattan_distance'] = (
    df['diff_bitterness'] + df['diff_sweetness'] +
    df['diff_acidity'] + df['diff_body']
)
```

3.4. taste_diff_mean

Опис: Середня різниця по всіх смакових характеристиках

Діапазон: 0.0 - 1.0

Код:

python

```
df['taste_diff_mean'] = df[['diff_bitterness', 'diff_sweetness',
                             'diff_acidity', 'diff_body']]
                             ].mean(axis=1)
```

3.5. `taste_weighted_similarity`

Опис: Зважена схожість (пріоритет на гіркоту та солодкість)

Діапазон: 0.0 - 1.0

Чому важливо: Гіркота та солодкість найважливіші для кави

Код:

```
python

df['taste_weighted_similarity'] = (
    0.3 * (1 - df['diff_bitterness']) +
    0.3 * (1 - df['diff_sweetness']) +
    0.2 * (1 - df['diff_acidity']) +
    0.2 * (1 - df['diff_body'])
)
```

4. USER AGGREGATE FEATURES (6)

Агреговані характеристики смакових переваг користувача.

4.1. `user_taste_pref_sum`

Опис: Сума всіх смакових переваг користувача

Діапазон: 0.0 - 4.0

Чому важливо: Показує загальну інтенсивність смакових переваг

Код:

```
python

taste_cols = ['taste_pref_bitterness', 'taste_pref_sweetness',
              'taste_pref_acidity', 'taste_pref_body']
users_df['user_taste_pref_sum'] = users_df[taste_cols].sum(axis=1)
```

4.2. `user_taste_pref_mean`

Опис: Середнє значення смакових переваг

Діапазон: 0.0 - 1.0

Код:

```
python

users_df['user_taste_pref_mean'] = users_df[taste_cols].mean(axis=1)
```

4.3. `user_taste_pref_std`

Опис: Стандартне відхилення смакових переваг

Діапазон: 0.0 - 0.5

Чому важливо: Показує наскільки різноманітні переваги користувача

Код:

```
python

users_df['user_taste_pref_std'] = users_df[taste_cols].std(axis=1)
```

4.4. `user_taste_pref_max`

Опис: Максимальна перевага

Діапазон: 0.0 - 1.0

Код:

```
python

users_df['user_taste_pref_max'] = users_df[taste_cols].max(axis=1)
```

4.5. `user_taste_pref_min`

Опис: Мінімальна перевага

Діапазон: 0.0 - 1.0

Код:

```
python

users_df['user_taste_pref_min'] = users_df[taste_cols].min(axis=1)
```

4.6. `user_dominant_taste_value`

Опис: Значення домінуючої смакової переваги

Діапазон: 0.0 - 1.0

Код:

```
python

users_df['user_dominant_taste_value'] = users_df[taste_cols].max(axis=1)
```

5. RECIPE AGGREGATE FEATURES (5)

Агреговані характеристики смакового профілю рецепту.

5.1. `recipe_taste_sum`

Опис: Сума всіх смакових характеристик рецепту

Діапазон: 0.0 - 4.0

Код:

```
python

taste_cols = ['taste_bitterness', 'taste_sweetness', 'taste_acidity', 'taste_body']
recipes_df['recipe_taste_sum'] = recipes_df[taste_cols].sum(axis=1)
```

5.2. recipe_taste_mean

Опис: Середнє значення смакових характеристик

Діапазон: 0.0 - 1.0

Код:

```
python

recipes_df['recipe_taste_mean'] = recipes_df[taste_cols].mean(axis=1)
```

5.3. recipe_taste_std

Опис: Стандартне відхилення смаків

Діапазон: 0.0 - 0.5

Чому важливо: Показує збалансованість смакового профілю

Код:

```
python

recipes_df['recipe_taste_std'] = recipes_df[taste_cols].std(axis=1)
```

5.4. recipe_taste_complexity

Опис: Складність смаку (обчислена через ентропію)

Діапазон: 0.0 - 1.386 (log(4))

Чому важливо: Висока ентропія = складний, багатогранний смак

Код:

```
python

from scipy.stats import entropy

recipes_df['recipe_taste_complexity'] = recipes_df[taste_cols].apply(
    lambda row: entropy(row + 0.01), axis=1
)
```

5.5. recipe_taste_balance

Опис: Збалансованість смаку (1 - std)

Діапазон: 0.5 - 1.0

Чому важливо: Високе значення = збалансований смак

Код:

```
python

recipes_df['recipe_taste_balance'] = 1 - recipes_df['recipe_taste_std']
```

6. EQUIPMENT FEATURES (12)

Фічі пов'язані з обладнанням користувача та вимогами рецепту.

6.1. `owned_equipment_count`

Опис: Кількість обладнання у користувача

Діапазон: 0 - 10+

Код:

```
python

users_df['owned_equipment_list'] = users_df['owned_equipment'].apply(
    lambda x: json.loads(x) if isinstance(x, str) else x
)
users_df['owned_equipment_count'] = users_df['owned_equipment_list'].apply(len)
```

6.2. `required_equipment_count`

Опис: Кількість необхідного обладнання для рецепту

Діапазон: 0 - 5

Код:

```
python

recipes_df['required_equipment_count'] = recipes_df['required_equipment_list'].apply(len)
```

6.3. `equipment_match`

Опис: Чи має користувач ВСЕ необхідне обладнання

Діапазон: 0 (немає) або 1 (має все)

Чому важливо: КРИТИЧНА фіча! Без обладнання рецепт неможливий

Код:

```
python
```

```
def check_equipment_match(row):
    user_equipment = set(row.get('owned_equipment_list', []))
    required_equipment = set(row.get('required_equipment_list', []))
    return int(required_equipment.issubset(user_equipment))

df['equipment_match'] = df.apply(check_equipment_match, axis=1)
```

6.4. equipment_coverage

Опис: Частка необхідного обладнання, яка є у користувача

Діапазон: 0.0 - 1.0

Код:

```
python

def compute_equipment_coverage(row):
    user_equipment = set(row.get('owned_equipment_list', []))
    required_equipment = set(row.get('required_equipment_list', []))
    if not required_equipment:
        return 1.0
    return len(required_equipment.intersection(user_equipment)) / len(required_equipment)

df['equipment_coverage'] = df.apply(compute_equipment_coverage, axis=1)
```

6.5. equipment_missing_count

Опис: Кількість відсутнього обладнання

Діапазон: 0 - 5

Код:

```
python

def compute_missing_equipment(row):
    user_equipment = set(row.get('owned_equipment_list', []))
    required_equipment = set(row.get('required_equipment_list', []))
    return len(required_equipment - user_equipment)

df['equipment_missing_count'] = df.apply(compute_missing_equipment, axis=1)
```

6.6. has_espresso_machine

Опис: Чи є у користувача еспресо-машина

Діапазон: 0 або 1

Чому важливо: Ключове обладнання для багатьох рецептів

Код:

```
python
```

```
users_df['has_espresso_machine'] = users_df['owned_equipment_list'].apply(  
    lambda x: int('espresso_machine' in x)  
)
```

6.7. **has_grinder**

Опис: Чи є у користувача кавомолка

Діапазон: 0 або 1

Код:

```
python  
  
users_df['has_grinder'] = users_df['owned_equipment_list'].apply(  
    lambda x: int('grinder' in x)  
)
```

6.8. **has_milk_frother**

Опис: Чи є у користувача спінювач молока

Діапазон: 0 або 1

Код:

```
python  
  
users_df['has_milk_frother'] = users_df['owned_equipment_list'].apply(  
    lambda x: int('milk_frother' in x)  
)
```

6.9. **requires_espresso_machine**

Опис: Чи потрібна еспресо-машина для рецепту

Діапазон: 0 або 1

Код:

```
python  
  
recipes_df['requires_espresso_machine'] = recipes_df['required_equipment_list'].apply(  
    lambda x: int('espresso_machine' in x)  
)
```

6.10. **requires_milk_frother**

Опис: Чи потрібен спінювач молока

Діапазон: 0 або 1

Код:

```
python
```

```
recipes_df['requires_milk_frother'] = recipes_df['required_equipment_list'].apply(
    lambda x: int('milk_frother' in x)
)
```

6.11. `equipment_sophistication_user`

Опис: Рівень складності обладнання користувача

Діапазон: 0 - 20+

Чому важливо: Показує досвідченість користувача

Код:

```
python

equipment_sophistication = {
    'espresso_machine': 3, 'grinder': 2, 'milk_frother': 2,
    'pour_over': 1, 'french_press': 1, 'kettle': 1,
    'moka_pot': 2, 'aeropress': 2, 'cold_brew_maker': 2
}

users_df['equipment_sophistication_user'] = users_df['owned_equipment_list'].apply(
    lambda equip_list: sum(equipment_sophistication.get(e, 1) for e in equip_list)
)
```

6.12. `equipment_sophistication_recipe`

Опис: Рівень складності обладнання для рецепту

Діапазон: 0 - 10

Код:

```
python

recipes_df['equipment_sophistication_recipe'] = recipes_df['required_equipment_list'].apply(
    lambda equip_list: sum(equipment_sophistication.get(e, 1) for e in equip_list)
)
```

7. TECHNICAL MATCH FEATURES (5)

Технічні характеристики відповідності.

7.1. `portion_size_match`

Опис: Чи збігається бажаний розмір порції з рецептом

Діапазон: 0 або 1

Код:

```
python
```

```
df['portion_size_match'] = (df['preferred_portion_size_ml'] == df['portion_size_ml']).astype(int)
```

7.2. `portion_size_diff`

Опис: Різниця в розмірі порції (мл)

Діапазон: 0 - 200+

Код:

```
python

df['portion_size_diff'] = abs(df['preferred_portion_size_ml'] - df['portion_size_ml'])
```

7.3. `portion_size_ratio`

Опис: Співвідношення розмірів порцій

Діапазон: 0.0 - 1.0

Код:

```
python

df['portion_size_ratio'] = df.apply(
    lambda row: min(
        row['preferred_portion_size_ml'] / (row['portion_size_ml'] + 1),
        row['portion_size_ml'] / (row['preferred_portion_size_ml'] + 1)
    ),
    axis=1
)
```

7.4. `prep_time_acceptable`

Опис: Чи прийнятний час приготування (≤ 20 хв)

Діапазон: 0 або 1

Код:

```
python

df['prep_time_acceptable'] = (df['preparation_time_minutes'] <= 20).astype(int)
```

7.5. `prep_time_ratio`

Опис: Відношення часу приготування до стандарту (20 хв)

Діапазон: 0.0 - 2.0+

Код:

```
python

df['prep_time_ratio'] = df['preparation_time_minutes'] / 20.0
```

8. TAG AND CATEGORY FEATURES (9)

Категоріальні характеристики рецептів з тегів.

8.1. `tags_count`

Опис: Кількість тегів у рецепті

Діапазон: 0 - 10

Код:

```
python  
  
recipes_df['tags_count'] = recipes_df['tags_list'].apply(len)
```

8.2. `is_hot`

Опис: Чи є рецепт гарячим

Діапазон: 0 або 1

Код:

```
python  
  
recipes_df['is_hot'] = recipes_df['tags_list'].apply(lambda x: int('hot' in x))
```

8.3. `is_cold`

Опис: Чи є рецепт холодним

Діапазон: 0 або 1

Код:

```
python  
  
recipes_df['is_cold'] = recipes_df['tags_list'].apply(lambda x: int('cold' in x))
```

8.4. `is_iced`

Опис: Чи є рецепт з льодом

Діапазон: 0 або 1

Код:

```
python  
  
recipes_df['is_iced'] = recipes_df['tags_list'].apply(lambda x: int('iced' in x))
```

8.5. is_classic

Опис: Чи є рецепт класичним

Діапазон: 0 або 1

Код:

```
python  
  
recipes_df['is_classic'] = recipes_df['tags_list'].apply(lambda x: int('classic' in x))
```

8.6. is_quick

Опис: Чи є рецепт швидким

Діапазон: 0 або 1

Код:

```
python  
  
recipes_df['is_quick'] = recipes_df['tags_list'].apply(lambda x: int('quick' in x))
```

8.7. is_specialty

Опис: Чи є рецепт спеціальним/унікальним

Діапазон: 0 або 1

Код:

```
python  
  
recipes_df['is_specialty'] = recipes_df['tags_list'].apply(lambda x: int('specialty' in x))
```

8.8. is_strong

Опис: Чи позначений рецепт як міцний

Діапазон: 0 або 1

Код:

```
python  
  
recipes_df['is_strong'] = recipes_df['tags_list'].apply(lambda x: int('strong' in x))
```

8.9. is_sweet

Опис: Чи позначений рецепт як солодкий

Діапазон: 0 або 1

Код:

```
python
```



```
recipes_df['is_sweet'] = recipes_df['tags_list'].apply(lambda x: int('sweet' in x))
```

9. DIETARY FEATURES (5)

Дієтичні обмеження та сумісність.

9.1. dietary_restrictions_count

Опис: Кількість дієтичних обмежень користувача

Діапазон: 0 - 5

Код:

```
python
```

```
users_df['dietary_restrictions_count'] = users_df['dietary_restrictions_list'].apply(len)
```

9.2. is_vegan

Опис: Чи є користувач веганом

Діапазон: 0 або 1

Код:

```
python
```

```
users_df['is_vegan'] = users_df['dietary_restrictions_list'].apply(
    lambda x: int('vegan' in x)
)
```

9.3. is_lactose_intolerant

Опис: Чи має користувач непереносимість лактози

Діапазон: 0 або 1

Код:

```
python
```

```
users_df['is_lactose_intolerant'] = users_df['dietary_restrictions_list'].apply(
    lambda x: int('lactose_intolerant' in x)
)
```

9.4. dietary_compatible

Опис: Чи сумісний рецепт з дієтичними обмеженнями

Діапазон: 0 або 1

Чому важливо: Критична фіча для користувачів з обмеженнями

Код:

python

```
def check_dietary_compatible(row):
    restrictions = row.get('dietary_restrictions_list', [])
    if not restrictions:
        return 1

    # Check vegan/dairy restrictions
    if any(r in ['vegan', 'dairy_free', 'lactose_intolerant'] for r in restrictions):
        if row.get('requires_milk', 0) == 1:
            return 0

    return 1

df['dietary_compatible'] = df.apply(check_dietary_compatible, axis=1)
```

9.5. requires_milk

Опис: Чи потрібне молоко для рецепту

Діапазон: 0 або 1

Код:

python

```
recipes_df['requires_milk'] = recipes_df['required_products_list'].apply(
    lambda x: int(any('milk' in str(k).lower() for k in x.keys()))
    if isinstance(x, dict) else 0
)
```

10. HISTORICAL FEATURES (13)

Історичні статистики з минулих взаємодій.

10.1. user_total_interactions

Опис: Загальна кількість взаємодій користувача

Діапазон: 0 - 1000+

Джерело: Агреговано з interactions_train

Код:

python

```
user_stats = train_df.groupby('user_id').agg({
    'interaction_id': 'count'
}).reset_index()
user_stats.columns = ['user_id', 'user_total_interactions']
```

10.2. `user_avg_rating`

Опис: Середній рейтинг, який ставить користувач

Діапазон: 1.0 - 5.0

Чому важливо: Показує загальну задоволеність користувача

Код:

```
python

user_stats = train_df.groupby('user_id')['rating'].mean()
user_stats = user_stats.fillna(3.0) # для користувачів без рейтингів
```

10.3. `user_rating_std`

Опис: Стандартне відхилення рейтингів користувача

Діапазон: 0.0 - 2.0

Чому важливо: Високе std = вибірковий користувач

Код:

```
python

user_stats = train_df.groupby('user_id')['rating'].std()
user_stats = user_stats.fillna(0.0)
```

10.4. `user_completion_rate`

Опис: Частка завершених рецептів користувачем

Діапазон: 0.0 - 1.0

Чому важливо: Показує залученість користувача

Код:

```
python

user_stats = train_df.groupby('user_id')['completed'].mean()
```

10.5. `user Rated_count`

Опис: Кількість рецептів, які користувач оцінив

Діапазон: 0 - 1000+

Код:

```
python
```

```
user_stats = train_df.groupby('user_id')['rating'].count()
```

10.6. `recipe_total_interactions`

Опис: Загальна кількість взаємодій з рецептом

Діапазон: 0 - 1000+

Джерело: Агреговано з interactions_train

Код:

```
python

recipe_stats = train_df.groupby('recipe_id').agg({
    'interaction_id': 'count'
}).reset_index()
recipe_stats.columns = ['recipe_id', 'recipe_total_interactions']
```

10.7. `recipe_avg_rating`

Опис: Середній рейтинг рецепту

Діапазон: 1.0 - 5.0

Чому важливо: Показує якість рецепту

Код:

```
python

recipe_stats = train_df.groupby('recipe_id')['rating'].mean()
recipe_stats = recipe_stats.fillna(3.0)
```

10.8. `recipe_rating_std`

Опис: Стандартне відхилення рейтингів рецепту

Діапазон: 0.0 - 2.0

Код:

```
python

recipe_stats = train_df.groupby('recipe_id')['rating'].std()
recipe_stats = recipe_stats.fillna(0.0)
```

10.9. `recipe_completion_rate`

Опис: Частка завершень рецепту

Діапазон: 0.0 - 1.0

Чому важливо: Високий completion rate = надійний рецепт

Код:

```
python
```

```
recipe_stats = train_df.groupby('recipe_id')['completed'].mean()
```

10.10. `recipe_popularity_score`

Опис: Нормалізована популярність рецепту

Діапазон: 0.0 - 1.0

Чому важливо: Важливо для cold start

Код:

```
python

max_interactions = recipe_stats['recipe_total_interactions'].max()
recipe_stats['recipe_popularity_score'] = (
    recipe_stats['recipe_total_interactions'] / max_interactions
)
```

10.11. `user_tried_similar_count`

Опис: Кількість подібних рецептів, які пробував користувач

Діапазон: 0 - 100

Код:

```
python

# Placeholder - потребує більш складної логіки
df['user_tried_similar_count'] = 0
```

10.12. `user_avg_rating_similar`

Опис: Середній рейтинг подібних рецептів

Діапазон: 1.0 - 5.0

Код:

```
python

# Placeholder
df['user_avg_rating_similar'] = df['user_avg_rating']
```

10.13. `similar_recipe_completion_rate`

Опис: Completion rate подібних рецептів

Діапазон: 0.0 - 1.0

Код:

```
python
```

```
# Placeholder
```

```
df['similar_recipe_completion_rate'] = df['recipe_completion_rate']
```

11. COLD START FEATURES (2)

Фічі для нових користувачів без історії.

11.1. `recipe_global_popularity`

Опис: Глобальна популярність рецепту

Діапазон: 0.0 - 1.0

Чому важливо: Для cold users рекомендуємо популярні рецепти

Код:

```
python
```

```
recipe_stats['recipe_global_popularity'] = recipe_stats['recipe_popularity_score']
```

11.2. `recipe_category_popularity`

Опис: Популярність в категорії

Діапазон: 0.0 - 1.0

Код:

```
python
```

```
# Placeholder
```

```
df['recipe_category_popularity'] = df['recipe_popularity_score']
```

12. BASIC INTERACTION FEATURES (3)

Базові взаємодії між фічами.

12.1. `taste_match_x_equipment`

Опис: Смаковий збіг × наявність обладнання

Діапазон: 0.0 - 1.0

Чому важливо: Хороший смак БЕЗ обладнання = марно

Код:

```
python
```

```
df['taste_match_x_equipment'] = df['taste_cosine_similarity'] * df['equipment_match']
```

12.2. `taste_match_x_popularity`

Опис: Смаковий збіг × популярність

Діапазон: 0.0 - 1.0

Чому важливо: Популярні рецепти, які подобаються = win-win

Код:

```
python
```

```
df['taste_match_x_popularity'] = df['taste_cosine_similarity'] * df['recipe_popularity_score']
```

12.3. `strength_match_x_completion`

Опис: Збіг міцності × completion rate

Діапазон: 0.0 - 1.0

Код:

```
python
```

```
df['strength_match_x_completion'] = df['is_strength_match'] * df['recipe_completion_rate']
```

13. TEMPORAL FEATURES (9)

Коли користувач п'є каву (час доби, день тижня).

13.1. `user_pct_morning`

Опис: Частка ранкових взаємодій (6-11 год)

Діапазон: 0.0 - 1.0

Чому важливо: Люди п'ють різну каву вранці vs вечором

Джерело: Обчислено з timestamp в interactions_train

Код:

```
python
```

```
train_full['timestamp'] = pd.to_datetime(train_full['timestamp'])
train_full['hour'] = train_full['timestamp'].dt.hour

def get_time_of_day(hour):
    if 6 <= hour < 11:
        return 'morning'
    elif 11 <= hour < 17:
        return 'afternoon'
    elif 17 <= hour < 22:
        return 'evening'
    else:
        return 'night'

train_full['time_of_day'] = train_full['hour'].apply(get_time_of_day)

user_time_dist = train_full.groupby(['user_id', 'time_of_day']).size().unstack(fill_value=0)
user_time_dist = user_time_dist.div(user_time_dist.sum(axis=1), axis=0)
user_time_dist.columns = [f'user_pct_{col}' for col in user_time_dist.columns]
```

13.2. `user_pct_afternoon`

Опис: Частка денних взаємодій (11-17 год)

Діапазон: 0.0 - 1.0

Код: (див. вище)

13.3. `user_pct_evening`

Опис: Частка вечірніх взаємодій (17-22 год)

Діапазон: 0.0 - 1.0

Код: (див. вище)

13.4. `user_pct_night`

Опис: Частка нічних взаємодій (22-6 год)

Діапазон: 0.0 - 1.0

Код: (див. вище)

13.5. `user_dominant_time_encoded`

Опис: Домінуючий час доби (числовий код)

Діапазон: 0 (morning), 1 (afternoon), 2 (evening), 3 (night)

Код:

python


```
time_encoding = {
    'user_pct_morning': 0,
    'user_pct_afternoon': 1,
    'user_pct_evening': 2,
    'user_pct_night': 3
}
dominant_time = user_time_dist.idxmax(axis=1)
user_time_dist['user_dominant_time_encoded'] = dominant_time.map(time_encoding)
```

13.6. `user_pct_weekend`

Опис: Частка взаємодій у вихідні

Діапазон: 0.0 - 1.0

Чому важливо: У вихідні люди експериментують більше

Код:

```
python

train_full['day_of_week'] = train_full['timestamp'].dt.dayofweek
train_full['is_weekend'] = train_full['day_of_week'].isin([5, 6]).astype(int)
user_weekend = train_full.groupby('user_id')['is_weekend'].mean()
```

13.7. `user_time_consistency`

Опис: Наскільки стабільний користувач у часі (inverse entropy)

Діапазон: 0.0 - 1.0

Чому важливо: Високе значення = рутинний користувач

Код:

```
python

from scipy.stats import entropy

def calculate_time_consistency(row):
    probs = row[['user_pct_morning', 'user_pct_afternoon',
                 'user_pct_evening', 'user_pct_night']].values
    ent = entropy(probs + 0.001)
    max_entropy = np.log(4)
    return 1 - (ent / max_entropy)

user_time_dist['user_time_consistency'] = user_time_dist.apply(
    calculate_time_consistency, axis=1
)
```

13.8. `user_avg_hour_sin`

Опис: Синус середньої години (циклічне кодування)

Діапазон: -1.0 - 1.0

Чому важливо: Години циклічні (23 год близька до 0 год)

Код:

```
python

user_avg_hour = train_full.groupby('user_id')['hour'].mean()
user_time_dist['user_avg_hour_sin'] = np.sin(2 * np.pi * user_avg_hour / 24)
```

13.9. `user_avg_hour_cos`

Опис: Косинус середньої години

Діапазон: -1.0 - 1.0

Код:

```
python

user_time_dist['user_avg_hour_cos'] = np.cos(2 * np.pi * user_avg_hour / 24)
```

14. BEHAVIORAL FEATURES (6)

Що користувач ФАКТИЧНО споживає (не що говорить, а що робить).

14.1. `user_avg_strength_consumed`

Опис: Середня міцність кави, яку ФАКТИЧНО п'є користувач

Діапазон: 1.0 - 5.0

Чому важливо: КРИТИЧНА ФІЧА! Краще за stated preference!

Джерело: Обчислено з історії interactions × recipes

Код:

```
python

# Merge recipes to interactions
train_full = train_df.merge(
    recipes_df[['recipe_id', 'strength']],
    on='recipe_id'
)

user_strength_stats = train_full.groupby('user_id')['strength'].agg(['mean', 'std'])
user_strength_stats.columns = ['user_avg_strength_consumed', 'user_strength_variety']
```

14.2. `user_strength_variety`

Опис: Варіативність міцності (std)

Діапазон: 0.0 - 2.0

Чому важливо: Низьке std = користувач стабільний у перевагах

Код: (див. вище)

14.3. `user_avg_difficulty_consumed`

Опис: Середня складність рецептів, які готує користувач

Діапазон: 1.0 - 3.0

Код:

```
python

difficulty_mapping = {'beginner': 1, 'intermediate': 2, 'advanced': 3}
recipes_df['difficulty_numeric'] = recipes_df['difficulty'].map(difficulty_mapping)

train_full = train_full.merge(recipes_df[['recipe_id', 'difficulty_numeric']], on='recipe_id')
user_difficulty = train_full.groupby('user_id')['difficulty_numeric'].mean()
```

14.4. `user_avg_prep_time_consumed`

Опис: Середній час приготування рецептів користувача

Діапазон: 5 - 30 хвилин

Код:

```
python

user_prep_time = train_full.groupby('user_id')['preparation_time_minutes'].mean()
```

14.5. `user_avg_portion_size_consumed`

Опис: Середній розмір порції, яку п'є користувач

Діапазон: 150 - 500 ml

Код:

```
python

user_portion = train_full.groupby('user_id')['portion_size_ml'].mean()
```

14.6. `user_exploration_ratio`

Опис: Частка унікальних рецептів (exploration vs exploitation)

Діапазон: 0.0 - 1.0

Чому важливо: Високе = explorer, низьке = exploiter

Код:

```
python
```

```
user_unique_recipes = train_full.groupby('user_id')['recipe_id'].nunique()
user_total = train_full.groupby('user_id').size()
user_exploration = user_unique_recipes / user_total
```

15. ALIGNMENT FEATURES (2)

Наскільки stated preferences відповідають consumed behavior.

15.1. `user_strength_alignment`

Опис: Наскільки stated strength відповідає consumed strength

Діапазон: 0.0 - 1.0

Чому важливо: Низьке значення = не довіряти stated preference!

Код:

```
python

users_with_consumed = users_df[['user_id', 'preferred_strength']].merge(
    user_strength_stats[['user_id', 'user_avg_strength_consumed']],
    on='user_id'
)

user_strength_alignment = abs(
    users_with_consumed['preferred_strength'] -
    users_with_consumed['user_avg_strength_consumed']
)

user_strength_alignment = (5 - user_strength_alignment) / 5
```

15.2. `user_portion_alignment`

Опис: Наскільки stated portion відповідає consumed portion

Діапазон: 0.0 - 1.0

Код:

```
python
```

```
portion_mapping = {'small': 150, 'medium': 250, 'large': 350}
users_df['preferred_portion_size_ml'] = users_df['preferred_portion_size'].map(portion_mapping)

users_with_portion = users_df[['user_id', 'preferred_portion_size_ml']].merge(
    user_portion, on='user_id'
)

user_portion_alignment = abs(
    users_with_portion['preferred_portion_size_ml'] -
    users_with_portion['user_avg_portion_size_consumed']
)
user_portion_alignment = (200 - user_portion_alignment) / 200
```

16. ADVANCED CROSS FEATURES (47)

Складні взаємодії між фічами для покращення NDCG@5.

16.1. `taste_intensity_cross`

Опис: Інтенсивність смаку користувача × інтенсивність рецепту

Діапазон: 0.0 - 16.0

Чому важливо: Інтенсивні користувачі люблять інтенсивні рецепти

Код:

```
python

df['taste_intensity_cross'] = df['user_taste_pref_sum'] * df['recipe_taste_sum']
```

16.2. `dominant_taste_alignment`

Опис: Чи збігається домінуючий смак користувача з домінуючим смаком рецепту

Діапазон: 0 або 1

Чому важливо: КЛЮЧОВА для персоналізації! Якщо користувач любить гіркоту, рецепт повинен бути гірким

Код:

```
python
```

```
def check_dominant_taste_match(row):
    user_tastes = {
        'bitterness': row['taste_pref_bitterness'],
        'sweetness': row['taste_pref_sweetness'],
        'acidity': row['taste_pref_acidity'],
        'body': row['taste_pref_body']
    }
    recipe_tastes = {
        'bitterness': row['taste_bitterness'],
        'sweetness': row['taste_sweetness'],
        'acidity': row['taste_acidity'],
        'body': row['taste_body']
    }
    user_dominant = max(user_tastes, key=user_tastes.get)
    recipe_dominant = max(recipe_tastes, key=recipe_tastes.get)
    return int(user_dominant == recipe_dominant)

df['dominant_taste_alignment'] = df.apply(check_dominant_taste_match, axis=1)
```

16.3. `taste_complexity_match`

Опис: Наскільки збігається складність смакових переваг

Діапазон: 0.0 - 1.0

Чому важливо: Користувачі зі складними перевагами люблять складні рецепти

Код:

```
python

df['taste_complexity_match'] = 1 - abs(df['user_taste_pref_std'] - df['recipe_taste_std'])
```

16.4. `morning_strength_score`

Опис: Ранковий час × міцність кави

Діапазон: 0.0 - 1.0

Чому важливо: Люди п'ють міцнішу каву вранці

Код:

```
python

df['morning_strength_score'] = df['user_pct_morning'] * (df['strength'] / 5.0)
```

16.5. `evening_specialty_score`

Опис: Вечірній час × спеціальний рецепт

Діапазон: 0.0 - 1.0

Чому важливо: Вечором люди пробують спеціальні напої

Код:

```
python

df['evening_specialty_score'] = df['user_pct_evening'] * df['is_specialty']
```

16.6. **weekend_exploration_score**

Опис: Вихідні × схильність до експериментів × рідкісність рецепту

Діапазон: 0.0 - 1.0

Чому важливо: У вихідні explorers пробують нові рецепти

Код:

```
python

df['weekend_exploration_score'] = (
    df['user_pct_weekend'] *
    df['user_exploration_ratio'] *
    (1 - df['recipe_popularity_score'])
)
```

16.7. **consistency_difficulty_cross**

Опис: Часова стабільність × простота рецепту

Діапазон: 0.0 - 1.0

Чому важливо: Стабільні користувачі віддають перевагу простим рецептам

Код:

```
python

df['consistency_difficulty_cross'] = df['user_time_consistency'] * (1 - df['difficulty_numeric'] / 3.0)
```

16.8. **equipment_sophistication_match**

Опис: Наскільки збігається рівень обладнання

Діапазон: 0.0 - 1.0

Чому важливо: Користувачі з простим обладнанням не люблять складні рецепти

Код:

```
python
```

```

max_sophistication = max(
    df['equipment_sophistication_user'].max(),
    df['equipment_sophistication_recipe'].max(),
    1
)
df['equipment_sophistication_match'] = 1 - abs(
    df['equipment_sophistication_user'] -
    df['equipment_sophistication_recipe']
) / max_sophistication

```

16.9. **equipment_enables_difficulty**

Опис: Наявність обладнання × складність

Діапазон: 0.0 - 1.0

Чому важливо: Обладнання робить складні рецепти доступними

Код:

```

python

df['equipment_enables_difficulty'] = df['equipment_match'] * df['difficulty_numeric'] / 3.0

```

16.10. **equipment_specialty_affinity**

Опис: Кількість обладнання × спеціальний рецепт

Діапазон: 0.0 - 1.0

Чому важливо: Більше обладнання = більше спеціальних рецептів

Код:

```

python

df['equipment_specialty_affinity'] = (df['owned_equipment_count'] / 10.0) * df['is_specialty']

```

16.11. **consumed_strength_match**

Опис: Наскільки міцність рецепту відповідає ФАКТИЧНО спожитій міцності

Діапазон: 0.0 - 1.0

Чому важливо: ★ КРИТИЧНА! Набагато краща за stated preference!

Код:

```

python

df['consumed_strength_match'] = 1 - abs(df['user_avg_strength_consumed'] - df['strength']) / 5.0

```

16.12. **consumed_difficulty_match**

Опис: Наскільки складність відповідає ФАКТИЧНО готованим рецептам

Діапазон: 0.0 - 1.0

Код:

```
python
```

```
df['consumed_difficulty_match'] = 1 - abs(df['user_avg_difficulty_consumed'] - df['difficulty_numeric']) / 3.0
```

16.13. consumed_portion_match

Опис: Наскільки розмір порції відповідає ФАКТИЧНО спожитим

Діапазон: 0.0 - 1.0

Код:

```
python
```

```
df['consumed_portion_match'] = 1 - abs(df['user_avg_portion_size_consumed'] - df['portion_size_ml']) / 200.0
```

16.14. prep_time_comfort

Опис: Наскільки час приготування відповідає звичному

Діапазон: 0.0 - 1.0

Чому важливо: Користувачі прагнуть до звичного часу

Код:

```
python
```

```
df['prep_time_comfort'] = 1 - abs(df['user_avg_prep_time_consumed'] - df['preparation_time_minutes']) / 30.0
```

16.15. strength_alignment_x_rating

Опис: Alignment × середній рейтинг користувача

Діапазон: 0.0 - 1.0

Чому важливо: Aligned користувачі з високими рейтингами = надійні

Код:

```
python
```

```
df['strength_alignment_x_rating'] = df['user_strength_alignment'] * df['user_avg_rating'] / 5.0
```

16.16. portion_alignment_x_completion

Опис: Portion alignment × completion rate

Діапазон: 0.0 - 1.0

Код:

```
python
```

```
df['portion_alignment_x_completion'] = df['user_portion_alignment'] * df['user_completion_rate']
```

16.17. `overall_alignment_score`

Опис: Загальний alignment score (зважене)

Діапазон: 0.0 - 1.0

Чому важливо: Комплексна метрика alignment

Код:

```
python

df['overall_alignment_score'] = (
    df['user_strength_alignment'] * 0.6 +
    df['user_portion_alignment'] * 0.4
)
```

16.18. `popular_aligned_score`

Опис: Популярність × смаковий збіг

Діапазон: 0.0 - 1.0

Чому важливо: Best of both worlds - популярні рецепти, які подобаються

Код:

```
python

df['popular_aligned_score'] = df['recipe_popularity_score'] * df['taste_cosine_similarity']
```

16.19. `reliable_taste_match`

Опис: Completion rate × смаковий збіг

Діапазон: 0.0 - 1.0

Чому важливо: Надійні рецепти, які подобаються

Код:

```
python

df['reliable_taste_match'] = df['recipe_completion_rate'] * df['taste_cosine_similarity']
```

16.20. `quality_fit_score`

Опис: Рейтинг рецепту × смаковий збіг

Діапазон: 0.0 - 1.0

Чому важливо: Якісні рецепти + персоналізація

Код:

```
python

df['quality_fit_score'] = (df['recipe_avg_rating'] / 5.0) * df['taste_cosine_similarity']
```

16.21. `exploration_novelty_score`

Опис: Exploration ratio × рідкісність рецепту

Діапазон: 0.0 - 1.0

Чому важливо: Explorers люблять нові рецепти

Код:

```
python

df['exploration_novelty_score'] = df['user_exploration_ratio'] * (1 - df['recipe_popularity_score'])
```

16.22. `hot_morning_score`

Опис: Гарячий напій × ранковий час

Діапазон: 0.0 - 1.0

Чому важливо: Вранці люди п'ють гарячі напої

Код:

```
python

df['hot_morning_score'] = df['is_hot'] * df['user_pct_morning']
```

16.23. `cold_afternoon_score`

Опис: Холодний напій × денний час

Діапазон: 0.0 - 1.0

Чому важливо: Вдень популярна холодна кава

Код:

```
python

df['cold_afternoon_score'] = df['is_cold'] * df['user_pct_afternoon']
```

16.24. `quick_activity_score`

Опис: Швидкий рецепт × активність користувача

Діапазон: 0.0 - 1.0

Чому важливо: Активні користувачі віддають перевагу швидким рецептам

Код:

```
python

df['quick_activity_score'] = df['is_quick'] * (df['user_total_interactions'] / 100.0)
```

16.25. `classic_consistency_score`

Опис: Класичний рецепт × часова стабільність

Діапазон: 0.0 - 1.0

Чому важливо: Стабільні користувачі люблять класику

Код:

```
python

df['classic_consistency_score'] = df['is_classic'] * df['user_time_consistency']
```

16.26. **sweet_preference_match**

Опис: Солодкий тег × низька перевага гіркоти

Діапазон: 0.0 - 1.0

Код:

```
python

df['sweet_preference_match'] = df['is_sweet'] * (1 - df['taste_pref_bitterness'])
```

16.27. **strong_tag_strength_match**

Опис: Тег "strong" × збіг міцності

Діапазон: 0.0 - 1.0

Код:

```
python

df['strong_tag_strength_match'] = df['is_strong'] * df['is_strength_match']
```

16.28. **vegan_compatible_taste**

Опис: Vegan × no milk × смаковий збіг

Діапазон: 0.0 - 1.0

Чому важливо: Для веганів критично важливо

Код:

```
python

df['vegan_compatible_taste'] = df['is_vegan'] * (1 - df['requires_milk']) * df['taste_cosine_similarity']
```

16.29. **restricted_simple_score**

Опис: Дієтичні обмеження × простота рецепту

Діапазон: 0.0 - 1.0

Чому важливо: Користувачі з обмеженнями віддають перевагу простим рецептам

Код:

```
python
```

```
df['restricted_simple_score'] = (df['dietary_restrictions_count'] / 5.0) * (1 - df['recipe_taste_complexity'])
```

16.30. `user_experience_level`

Опис: Рівень досвіду (log-нормалізована кількість взаємодій)

Діапазон: 0.0 - 1.0+

Код:

```
python

df['user_experience_level'] = np.log1p(df['user_total_interactions']) / 5.0
```

16.31. `experience_difficulty_match`

Опис: Досвід × складність

Діапазон: 0.0 - 1.0

Чому важливо: Досвідчені користувачі можуть готувати складні рецепти

Код:

```
python

df['experience_difficulty_match'] = df['user_experience_level'] * (df['difficulty_numeric'] / 3.0)
```

16.32. `experienced_explorer`

Опис: Досвід × exploration ratio

Діапазон: 0.0 - 1.0+

Код:

```
python

df['experienced_explorer'] = df['user_experience_level'] * df['user_exploration_ratio']
```

16.33. `taste_match_squared`

Опис: Квадрат смакового збігу

Діапазон: 0.0 - 1.0

Чому важливо: ★ HIGH IMPACT! Амплі

фікує хороші збіги

Код:

```
python

df['taste_match_squared'] = df['taste_cosine_similarity'] ** 2
```

16.34. `equipment_taste_squared`

Опис: Обладнання × квадрат смакового збігу

Діапазон: 0.0 - 1.0

Чому важливо: ★ КРИТИЧНА КОМБО! Обладнання + відмінний смак

Код:

```
python

df['equipment_taste_squared'] = df['equipment_match'] * (df['taste_cosine_similarity'] ** 2)
```

16.35. `alignment_consumed_product`

Опис: Overall alignment × consumed strength match

Діапазон: 0.0 - 1.0

Код:

```
python

df['alignment_consumed_product'] = df['overall_alignment_score'] * df['consumed_strength_match']
```

16.36. `rating_completion_taste`

Опис: Рейтинг × completion × смаковий збіг (потрійна взаємодія)

Діапазон: 0.0 - 1.0

Чому важливо: Комплексна метрика якості

Код:

```
python

df['rating_completion_taste'] = (
    (df['user_avg_rating'] / 5.0) *
    df['user_completion_rate'] *
    df['taste_cosine_similarity']
)
```

16.37. `stated_consumed_strength_ratio`

Опис: Відношення stated до consumed strength

Діапазон: 0.2 - 5.0

Чому важливо: Показує наскільки stated preference відхиляється

Код:

```
python

df['stated_consumed_strength_ratio'] = df['preferred_strength'] / (df['user_avg_strength_consumed'] + 0.1)
```

16.38. `picky_user_good_recipe`

Опис: Вибірковість користувача × якість рецепту

Діапазон: 0.0 - 2.0

Чому важливо: Вибірковим користувачам потрібні якісні рецепти

Код:

```
python

df['picky_user_good_recipe'] = df['user_rating_std'] * (df['recipe_avg_rating'] / 5.0)
```

16.39. `popularity_momentum`

Опис: Популярність × кількість оцінок

Діапазон: 0.0 - 1.0

Чому важливо: Швидко зростаючі рецепти

Код:

```
python

df['popularity_momentum'] = df['recipe_popularity_score'] * df['recipe_rated_count'] / 100.0
```

16.40. `morning_combo_score`

Опис: Ранок × міцність × гарячий

Діапазон: 0.0 - 1.0

Чому важливо: Класична ранкова комбінація

Код:

```
python

df['morning_combo_score'] = df['user_pct_morning'] * (df['strength'] / 5.0) * df['is_hot']
```

16.41. `afternoon_combo_score`

Опис: День × солодкий × холодний

Діапазон: 0.0 - 1.0

Чому важливо: Денне задоволення

Код:

```
python

df['afternoon_combo_score'] = df['user_pct_afternoon'] * df['is_sweet'] * df['is_cold']
```

16.42. `evening_combo_score`

Опис: Вечір × спеціальний × складний

Діапазон: 0.0 - 1.386

Чому важливо: Вечірнє задоволення

Код:

```
python

df['evening_combo_score'] = df['user_pct_evening'] * df['is_specialty'] * df['recipe_taste_complexity']
```

16.43. **weekend_project_score**

Опис: Вихідні × складність × exploration

Діапазон: 0.0 - 1.0

Чому важливо: У вихідні час для складних проєктів

Код:

```
python

df['weekend_project_score'] = (
    df['user_pct_weekend'] *
    (df['difficulty_numeric'] / 3.0) *
    df['user_exploration_ratio']
)
```

16.44. **multi_similarity_product**

Опис: Смак × consumed strength × consumed difficulty

Діапазон: 0.0 - 1.0

Чому важливо: Комплексна метрика схожості

Код:

```
python

df['multi_similarity_product'] = (
    df['taste_cosine_similarity'] *
    df['consumed_strength_match'] *
    df['consumed_difficulty_match']
)
```

16.45. **triple_alignment_score**

Опис: Смак × обладнання × дієта (всі повинні співпадати!)

Діапазон: 0.0 - 1.0

Чому важливо: ★ КРИТИЧНА! Якщо хоч щось не підходить = провал

Код:

```
python
```



```
df['triple_alignment_score'] = (  
    df['taste_cosine_similarity'] *  
    df['equipment_match'] *  
    df['dietary_compatible']  
)
```

16.46. **strength_consistency_score**

Опис: $(1 - \text{variety}) \times \text{strength match}$

Діапазон: 0.0 - 1.0

Чому важливо: Стабільні користувачі цінують точний збіг

Код:

```
python  
  
df['strength_consistency_score'] = (1 - df['user_strength_variety'] / 2.0) * df['is_strength_match']
```

16.47. **consistent_reliable_match**

Опис: Часова стабільність $\times$ completion rate

Діапазон: 0.0 - 1.0

Код:

```
python  
  
df['consistent_reliable_match'] = df['user_time_consistency'] * df['recipe_completion_rate']
```

16.48. **expert_rarity_affinity**

Опис: Досвід $\times$ рідкісність $\times$ складність

Діапазон: 0.0 - 1.0

Чому важливо: Експерти люблять рідкісні складні рецепти

Код:

```
python  
  
df['expert_rarity_affinity'] = (  
    df['user_experience_level'] *  
    (1 - df['recipe_popularity_score']) *  
    (df['difficulty_numeric'] / 3.0)  
)
```

16.49. **beginner_popular_affinity**

Опис: $(1 - \text{досвід}) \times \text{популярність} \times (1 - \text{складність})$

Діапазон: 0.0 - 1.0

Чому важливо: Новачки люблять популярні прості рецепти

Код:

```
python

df['beginner_popular_affinity'] = (
    (1 - df['user_experience_level']) *
    df['recipe_popularity_score'] *
    (1 - df['difficulty_numeric'] / 3.0)
)
```

17. MISCELLANEOUS (1)

17.1. **preference_mismatch**

Опис: Placeholder для розбіжності переваг

Діапазон: 0.0

Код:

```
python

df['preference_mismatch'] = 0 # Placeholder
```

ПІДСУМОК

Загальна кількість фіч: ~147

Категорії:

1. Basic Taste: 8
2. Taste Differences: 6
3. Advanced Taste Match: 5
4. User Aggregates: 6
5. Recipe Aggregates: 5
6. Equipment: 12
7. Technical Match: 5
8. Tags: 9
9. Dietary: 5
10. Historical: 13
11. Cold Start: 2

12. Basic Interactions: 3
13. Temporal: 9
14. Behavioral: 6
15. Alignment: 2
16. Advanced Cross: 47
17. Miscellaneous: 1

ТОП-10 найважливіших фіч для NDCG@5:

1. `equipment_match` - КРИТИЧНА! Без обладнання рецепт неможливий
2. `taste_cosine_similarity` - Головна метрика персоналізації
3. `consumed_strength_match` - Краще за stated preference
4. `dominant_taste_alignment` - Ключова для персоналізації
5. `equipment_taste_squared` - Потужна комбінація
6. `triple_alignment_score` - Всі фактори мають співпадати
7. `recipe_popularity_score` - Важливо для cold start
8. `user_avg_rating` - Показує задоволення
9. `dietary_compatible` - Критично для користувачів з обмеженнями
10. `morning_strength_score` - Часовий контекст важливий

Джерела даних:

- `users.csv`: 12 стовпців
- `recipes.csv`: 13 стовпців
- `interactions_train.csv`: 6 стовпців
- **Обчислені**: ~122 фічі

Використання:

```
python
```

1. Ініціалізація

```
fe = FeatureEngineer()
```

2. Fit на train data

```
fe.fit(users_df, recipes_df, train_interactions_df)
```

3. Generate features

```
features = fe.generate(candidates_df, users_df, recipes_df)
```

4. Train model

```
model.fit(features, labels)
```